

Лабораторная работа №3

Управляющие структуры

Доберштейн А. С.

Содержание

1	Цель работы	4
2	Выполнение лабораторной работы	5
3	Выводы	25

Список иллюстраций

2.1	Циклы while и for	5
2.2	Циклы while и for	5
2.3	Циклы while и for	6
2.4	Циклы while и for	6
2.5	Циклы while и for	7
2.6	Условные выражения	7
2.7	Функции	7
2.8	Функции	8
2.9	Функции	8
2.10	Функции	8
2.11	Функции	9
2.12	Функции	9
2.13	Функции	9
2.14	Функции	10
2.15	Функции	10
2.16	Сторонние библиотеки (пакеты) в Julia	11
2.17	Задание для самостоятельного выполнения №1	12
2.18	Задание для самостоятельного выполнения №1	13
2.19	Задание для самостоятельного выполнения №2	13
2.20	Задание для самостоятельного выполнения №3	14
2.21	Задание для самостоятельного выполнения №4	14
2.22	Задание для самостоятельного выполнения №5	15
2.23	Задание для самостоятельного выполнения №6	16
2.24	Задание для самостоятельного выполнения №7	17
2.25	Задание для самостоятельного выполнения №7	18
2.26	Задание для самостоятельного выполнения №7	18
2.27	Задание для самостоятельного выполнения №7	19
2.28	Задание для самостоятельного выполнения №8	20
2.29	Задание для самостоятельного выполнения №8	21
2.30	Задание для самостоятельного выполнения №8	21
2.31	Задание для самостоятельного выполнения №8	22
2.32	Задание для самостоятельного выполнения №9	22
2.33	Задание для самостоятельного выполнения №10	23
2.34	Задание для самостоятельного выполнения №10	23
2.35	Задание для самостоятельного выполнения №10	24
2.36	Задание для самостоятельного выполнения №11	24

1 Цель работы

Основная цель работы — освоить применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

2 Выполнение лабораторной работы

Повторим примеры из лабораторной работы. (рис. 2.1 - 2.16).

```
[1]:  
# Циклы while u for  
  
n = 0  
while n < 10  
    n += 1  
    println(n)  
end  
  
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

Рис. 2.1: Циклы while и for

```
myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]  
  
i = 1  
while i <= length(myfriends)  
    friend = myfriends[i]  
    println("Hi $friend, it's great to see you!")  
    i += 1  
end  
  
Hi Ted, it's great to see you!  
Hi Robyn, it's great to see you!  
Hi Barney, it's great to see you!  
Hi Lily, it's great to see you!  
Hi Marshall, it's great to see you!
```

Рис. 2.2: Циклы while и for

```

for n in 1:2:10
    println(n)
end
1
3
5
7
9

[4]:

myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]

for friend in myfriends
    println("Hi $friend, it's great to see you!")
end

Hi Ted, it's great to see you!
Hi Robyn, it's great to see you!
Hi Barney, it's great to see you!
Hi Lily, it's great to see you!
Hi Marshall, it's great to see you!

```

Рис. 2.3: Циклы while и for

```

# инициализация массива m x n из нулей:
m, n = 5, 5
A = fill{0, (m, n)}

# формирование массива, в котором значение каждой записи
# является суммой индексов строки и столбца:
for i in 1:m
    for j in 1:n
        A[i, j] = i + j
    end
end
A

[6]:

5x5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10

[8]:

B = fill{0, (m, n)}

for i in 1:m, j in 1:n
    B[i, j] = i + j
end
B

[8]:

5x5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10

```

Рис. 2.4: Циклы while и for

```
C = [i + j for i in 1:m, j in 1:n]
C
```

[9]:

```
5x5 Matrix{Int64}:
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
 5  6  7  8  9
 6  7  8  9 10
```

Рис. 2.5: Циклы while и for

[12]:

```
# Условные выражения
N = 15
# используем `&&` для реализации операции "AND"
# операция % вычисляет остаток от деления
if (N % 3 == 0) && (N % 5 == 0)
    println("FizzBuzz")
elseif N % 3 == 0
    println("Fizz")
elseif N % 5 == 0
    println("Buzz")
else
    println(N)
end
```

FizzBuzz

[13]:

```
x = 5
y = 10

(x > y) ? x : y
```

[13]:

10

Рис. 2.6: Условные выражения

[20]:

```
# Функции

function sayhi(name)
    println("Hi $name, it's great to see you!")
end

sayhi("C-3PO")
```

Hi C-3PO, it's great to see you!

[21]:

```
# функция возведения в квадрат:
function f(x)
    x^2
end

f(42)
```

[21]:

1764

Рис. 2.7: Функции

```
[23]:
sayhi2(name) = println("Hi $name, it's great to see you!")
sayhi2("C-3P0")
Hi C-3P0, it's great to see you!

[24]:
f2(x) = x^2
f2(42)

[24]:
1764
```

Рис. 2.8: Функции

```
[25]:
sayhi3 = name-> println("Hi $name, it's great to see you!")
sayhi3("C-3P0")
Hi C-3P0, it's great to see you!

[26]:
f3 = x-> x^2
f3(42)

[26]:
1764
```

Рис. 2.9: Функции

```
[28]:
v = [3, 5, 2]

[28]:
3-element Vector{Int64}:
 3
 5
 2

[29]:
sort(v)
v

[29]:
3-element Vector{Int64}:
 3
 5
 2

[30]:
sort!(v)
v

[30]:
3-element Vector{Int64}:
 2
 3
 5
```

Рис. 2.10: Функции


```
[31]:
map(f,[1, 2, 3])

[31]:
3-element Vector{Int64}:
 1
 4
 9

[32]:
map(x->x^3,[1,2, 3])

[32]:
3-element Vector{Int64}:
 1
 8
27
```

Рис. 2.11: Функции

```
f(x) = x^2
broadcast(f,[1,2, 3])

[33]:
3-element Vector{Int64}:
 1
 4
 9

[34]:
f.([1, 2, 3])

[34]:
3-element Vector{Int64}:
 1
 4
 9
```

Рис. 2.12: Функции

```
A = [i + 3*j for j in 0:2, i in 1:3]

[40]:
3×3 Matrix{Int64}:
 1  2  3
 4  5  6
 7  8  9

[41]:
f(A)

[41]:
3×3 Matrix{Int64}:
 30  36  42
 66  81  96
102 126 150
```

Рис. 2.13: Функции

```

B = f.(A)

[42]:
3×3 Matrix{Int64}:
 1  4  9
16 25 36
49 64 81

[43]:
broadcast(x-> x+ 2 * f(x) / x,A)

[43]:
3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

```

Рис. 2.14: Функции

```

@. A + 2 * f(A) / A

[44]:
3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

[45]:
A .+ 2 .*f.(A) ./A

[45]:
3×3 Matrix{Float64}:
 3.0  6.0  9.0
12.0 15.0 18.0
21.0 24.0 27.0

```

Рис. 2.15: Функции

```
import Pkg
Pkg.add("Colors")

[32m[1m Updating[22m[39m registry at `C:\Users\MI\.julia\registries\General.toml:

[47]:
Pkg.add("Colors")
using Colors

[32m[1m Resolving[22m[39m package versions... •••

[48]:
palette= distinguishable_colors(100)

[48]:


[50]:
rand(palette, 3,3)

[50]:

```

Рис. 2.16: Сторонние библиотеки (пакеты) в Julia

Выполним задания для самостоятельного выполнения. (рис. 2.17 - 2.36).

1. Используя циклы `while` и `for`:
 - выведите на экран целые числа от 1 до 100 и напечатайте их квадраты;
 - создайте словарь `squares`, который будет содержать целые числа в качестве ключей и квадраты в качестве их пар-значений;
 - создайте массив `squares_arr`, содержащий квадраты всех чисел от 1 до 100.

```
[53]:  
# 1  
  
n = 1  
  
while n <= 100  
    println(n, " ", n^2)  
    n += 1  
end
```

```
1  1  
2  4  
3  9  
4  16  
5  25  
6  36  
7  49  
8  64  
9  81  
10 100  
11 121  
12 144  
13 169  
14 196  
15 225  
16 256  
17 289  
18 324  
19 361  
20 400  
21 441  
22 484  
23 529
```

Рис. 2.17: Задание для самостоятельного выполнения №1

```
squares = Dict()

for i in 1:1:100
    squares[i] = i^2
end

print(squares)
```

Dict{Any, Any}(5 => 25, 56 => 3136, 35 => 1225, 55 => 3025, 60 => 3600, 30 => 900, 32 => 1024, 6 => 36, 67 => 4489, 45 => 2025, 73 => 5329, 64 => 4096, 90 => 8100, 4 => 16, 13 => 169, 54 => 2916, 63 => 3969, 86 => 7396, 91 => 8281, 62 => 3844, 58 => 3364, 52 => 2704, 12 => 144, 28 => 784, 75 => 5625, 23 => 529, 92 => 8464, 41 => 1681, 43 => 1849, 11 => 121, 36 => 1296, 68 => 4624, 69 => 4761, 98 => 9604, 82 => 6724, 85 => 7225, 39 => 1521, 84 => 7056, 77 => 5929, 7 => 49, 25 => 625, 95 => 9025, 71 => 5041, 66 => 4356, 76 => 5776, 34 => 1156, 50 => 2500, 59 => 3481, 93 => 8649, 2 => 4, 10 => 100, 18 => 324, 26 => 676, 27 => 729, 42 => 1764, 87 => 7569, 100 => 10000, 79 => 6241, 16 => 256, 20 => 400, 81 => 6561, 19 => 361, 49 => 2401, 44 => 1936, 9 => 81, 31 => 961, 74 => 5476, 61 => 3721, 29 => 841, 94 => 8836, 46 => 2116, 57 => 3249, 70 => 4900, 21 => 441, 38 => 1444, 88 => 7744, 78 => 6084, 72 => 5184, 24 => 576, 8 => 64, 17 => 289, 37 => 1369, 1 => 1, 53 => 2809, 22 => 484, 47 => 2209, 83 => 6889, 99 => 9801, 89 => 7921, 14 => 196, 3 => 9, 80 => 6400, 96 => 9216, 51 => 2601, 33 => 1089, 40 => 1600, 48 => 2304, 15 => 225, 65 => 4225, 97 => 9409)

[57]:

```
squares_arr = [n^2 for n in 1:100]

squares_arr
```

[57]:

100-element Vector{Int64}:

```
1
4
9
16
25
36
49
64
81
100
```

Рис. 2.18: Задание для самостоятельного выполнения №1

2. Напишите условный оператор, который печатает число, если число чётное, и строку «нечётное», если число нечётное. Перепишите код, используя тернарный оператор.

[59]:

```
# 2

n = 11

if n % 2 == 0
    println(n)
else
    println("Нечетное")
end
```

Нечетное

[60]:

```
print(n % 2 == 0 ? n : "Нечетное")
```

Нечетное

Рис. 2.19: Задание для самостоятельного выполнения №2

3. Напишите функцию `add_one`, которая добавляет 1 к своему входу.

```
[76]:  
# 3  
  
function add_one(x)  
    x += 1  
    return x  
end  
  
add_one(1), add_one(5), add_one(10)  
  
[76]:  
(2, 6, 11)
```

Рис. 2.20: Задание для самостоятельного выполнения №3

4. Используйте `map()` или `broadcast()` для задания матрицы A , каждый элемент которой увеличивается на единицу по сравнению с предыдущим.

```
[77]:  
# 4  
  
[map(x -> x += 1, i + 4 * j) for i = 0:3, j = 1:4]  
  
[77]:  
4x4 Matrix{Int64}:  
 5  9 13 17  
 6 10 14 18  
 7 11 15 19  
 8 12 16 20
```

Рис. 2.21: Задание для самостоятельного выполнения №4

5. Задайте матрицу A следующего вида:

$$A = \begin{pmatrix} 1 & 1 & 3 \\ 5 & 2 & 6 \\ -2 & -1 & -3 \end{pmatrix}.$$

- Найдите A^3 .
- Замените третий столбец матрицы A на сумму второго и третьего столбцов.

[82]:

```
# 5
```

```
A = [1 1 3; 5 2 6; -2 -1 -3]
```

[82]:

```
3x3 Matrix{Int64}:  
 1  1  3  
 5  2  6  
-2 -1 -3
```

[83]:

```
A^3, A.^3
```

[83]:

```
([0 0 0; 0 0 0; 0 0 0], [1 1 27; 125 8 216; -8 -1 -27])
```

[84]:

```
A[:, 3] = A[:, 2] + A[:, 3]  
A
```

[84]:

```
3x3 Matrix{Int64}:  
 1  1  4  
 5  2  8  
-2 -1 -4
```

Рис. 2.22: Задание для самостоятельного выполнения №5

6. Создайте матрицу B с элементами $B_{i1} = 10, B_{i2} = -10, B_{i3} = 10, i = 1, 2, \dots, 15$.
Вычислите матрицу $C = B^T B$.

[87]:

```
# 6
```

```
B = [10*(-1)^(j-1)) for i = 1:15, j = 1:3]
```

[87]:

```
15x3 Matrix{Int64}:
```

```
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
10  -10  10
```

[88]:

```
C = B' * B
```

[88]:

```
3x3 Matrix{Int64}:
```

```
1500  -1500  1500
-1500  1500  -1500
1500  -1500  1500
```

Рис. 2.23: Задание для самостоятельного выполнения №6

7. Создайте матрицу Z размерности 6×6 , все элементы которой равны нулю, и матрицу E , все элементы которой равны 1. Используя цикл `while` или `for` и закономерности расположения элементов, создайте следующие матрицы размерности 6×6 :

$$Z_1 = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \quad Z_2 = \begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \end{pmatrix},$$

$$Z_3 = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}, \quad Z_4 = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}.$$

[89]:

```
# 7
```

```
Z = zeros(Int, 6, 6)
Z
```

[89]:

```
6x6 Matrix{Int64}:
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
 0  0  0  0  0  0
```

Рис. 2.24: Задание для самостоятельного выполнения №7

```
E = ones(Int, 6, 6)
E
```

[90]:

```
6x6 Matrix{Int64}:
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
 1  1  1  1  1  1
```

[96]:

```
Z1 = copy(Z)

for i in 1:6
    for j in 1:6
        if abs(i - j) == 1
            Z1[i, j] = 1
        end
    end
end

Z1
```

[96]:

```
6x6 Matrix{Int64}:
 0  1  0  0  0  0
 1  0  1  0  0  0
 0  1  0  1  0  0
 0  0  1  0  1  0
 0  0  0  1  0  1
 0  0  0  0  1  0
```

Рис. 2.25: Задание для самостоятельного выполнения №7

[111]:

```
Z2 = copy(Z)

for i in 1:6
    for j in 1:6
        if i == j
            Z2[i, j] = 1
        elseif abs(i-j) == 2
            Z2[i, j] = 1
        else
            Z2[i, j] = 0
        end
    end
end

Z2
```

[111]:

```
6x6 Matrix{Int64}:
 1  0  1  0  0  0
 0  1  0  1  0  0
 1  0  1  0  1  0
 0  1  0  1  0  1
 0  0  1  0  1  0
 0  0  0  1  0  1
```

Рис. 2.26: Задание для самостоятельного выполнения №7

```
[118]:
Z3 = copy(Z2)

Z3 = Z3[:, end:-1:1]

Z3
```

```
[118]:
6x6 Matrix{Int64}:
 0  0  0  1  0  1
 0  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  0
 1  0  1  0  0  0
```

```
[119]:
Z4 = copy(Z)

for i in 1:6
    for j in 1:6
        if (i+j) % 2 == 0
            Z4[i, j] = 1
        end
    end
end

Z4
```

```
[119]:
6x6 Matrix{Int64}:
 1  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  1
 1  0  1  0  1  0
 0  1  0  1  0  1
```

Рис. 2.27: Задание для самостоятельного выполнения №7

8. В языке R есть функция `outer()`. Фактически, это матричное умножение с возможностью изменить применяемую операцию (например, заменить произведение на сложение или возведение в степень).

- Напишите свою функцию, аналогичную функции `outer()` языка R. Функция должна иметь следующий интерфейс: `outer(x, y, operation)`. Таким образом, функция вида `outer(A, B, *)` должна быть эквивалентна произведению матриц A и B размерностями $L \times M$ и $M \times N$ соответственно, где элементы результирующей матрицы C имеют вид $C_{ij} = \sum_{k=1}^M A_{ik} B_{kj}$ (или в тензорном виде $C_i^j = \sum_{k=1}^M A_k^i B_j^k$).
- Используя написанную вами функцию `outer()`, создайте матрицы следующей структуры:

$$A_1 = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 & 5 \\ 2 & 3 & 4 & 5 & 6 \\ 3 & 4 & 5 & 6 & 7 \\ 4 & 5 & 6 & 7 & 8 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 2 & 4 & 8 & 16 & 32 \\ 3 & 9 & 27 & 81 & 243 \\ 4 & 16 & 64 & 256 & 1024 \end{pmatrix}.$$

$$A_3 = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 \\ 1 & 2 & 3 & 4 & 0 \\ 2 & 3 & 4 & 0 & 1 \\ 3 & 4 & 0 & 1 & 2 \\ 4 & 0 & 1 & 2 & 3 \end{pmatrix}, \quad A_4 = \begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 8 & 9 & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 \\ 9 & 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \end{pmatrix},$$

$$A_5 = \begin{pmatrix} 0 & 8 & 7 & 6 & 5 & 4 & 3 & 2 & 1 \\ 1 & 0 & 8 & 7 & 6 & 5 & 4 & 3 & 2 \\ 2 & 1 & 0 & 8 & 7 & 6 & 5 & 4 & 3 \\ 3 & 2 & 1 & 0 & 8 & 7 & 6 & 5 & 4 \\ 4 & 3 & 2 & 1 & 0 & 8 & 7 & 6 & 5 \\ 5 & 4 & 3 & 2 & 1 & 0 & 8 & 7 & 6 \\ 6 & 5 & 4 & 3 & 2 & 1 & 0 & 8 & 7 \\ 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 & 8 \\ 8 & 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \end{pmatrix}.$$

В каждом случае ваше решение должно быть легко обобщаемым на случай создания матриц большей размерности, но той же структуры.

Рис. 2.28: Задание для самостоятельного выполнения №8

```
# 8
function outer(x, y, operation)
    return [operation(xi, yj) for xi in x, yj in y]
end
```

[120]:

outer (generic function with 1 method)

[121]:

```
A1 = outer(0:4, 0:4, +)
```

[121]:

```
5x5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  5
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8
```

[124]:

```
A2 = outer(0:4, 1:5, ^)
```

[124]:

```
5x5 Matrix{Int64}:
 0  0  0  0  0
 1  1  1  1  1
 2  4  8  16  32
 3  9  27  81  243
 4  16  64  256  1024
```

Рис. 2.29: Задание для самостоятельного выполнения №8

```
A3 = outer(0:4, 0:4, (x, y) -> mod(x+y, 5))
```

[127]:

```
5x5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  0
 2  3  4  0  1
 3  4  0  1  2
 4  0  1  2  3
```

[128]:

```
A4 = outer(0:9, 0:9, (x, y) -> mod(x+y, 10))
```

[128]:

```
10x10 Matrix{Int64}:
 0  1  2  3  4  5  6  7  8  9
 1  2  3  4  5  6  7  8  9  0
 2  3  4  5  6  7  8  9  0  1
 3  4  5  6  7  8  9  0  1  2
 4  5  6  7  8  9  0  1  2  3
 5  6  7  8  9  0  1  2  3  4
 6  7  8  9  0  1  2  3  4  5
 7  8  9  0  1  2  3  4  5  6
 8  9  0  1  2  3  4  5  6  7
 9  0  1  2  3  4  5  6  7  8
```

Рис. 2.30: Задание для самостоятельного выполнения №8

```
[129]:
```

```
A5 = outer(0:8, 0:8, (x, y) -> mod(x-y, 9))
```

```
[129]:
```

```
9x9 Matrix{Int64}:
```

```
0 8 7 6 5 4 3 2 1
1 0 8 7 6 5 4 3 2
2 1 0 8 7 6 5 4 3
3 2 1 0 8 7 6 5 4
4 3 2 1 0 8 7 6 5
5 4 3 2 1 0 8 7 6
6 5 4 3 2 1 0 8 7
7 6 5 4 3 2 1 0 8
8 7 6 5 4 3 2 1 0
```

Рис. 2.31: Задание для самостоятельного выполнения №8

9. Решите следующую систему линейных уравнений с 5 неизвестными:

$$\begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5 = 7, \\ 2x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 = -1, \\ 3x_1 + 2x_2 + x_3 + 2x_4 + 3x_5 = -3, \\ 4x_1 + 3x_2 + 2x_3 + x_4 + 2x_5 = 5, \\ 5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 = 17, \end{cases}$$

рассмотрев соответствующее матричное уравнение $Ax = y$. Обратите внимание на особый вид матрицы A . Метод, используемый для решения данной системы уравнений, должен быть легко обобщаем на случай большего числа уравнений, где матрица A будет иметь такую же структуру.

```
[131]:
```

```
# 9
```

```
A = [1 2 3 4 5;
      2 1 2 3 4;
      3 2 1 2 3;
      4 3 2 1 2;
      5 4 3 2 1]

b = [7; -1; -3; 5; 17];
```

```
[132]:
```

```
x = A \ b
print(x)
```

```
[-2.000000000000036, 3.000000000000058, 4.999999999999998, 1.999999999999991, -3.999999999999999]
```

Рис. 2.32: Задание для самостоятельного выполнения №9

10. Создайте матрицу M размерности 6×10 , элементами которой являются целые числа, выбранные случайным образом с повторениями из совокупности $1, 2, \dots, 10$.
- Найдите число элементов в каждой строке матрицы M , которые больше числа N (например, $N = 4$).
 - Определите, в каких строках матрицы M число M (например, $M = 7$) встречается ровно 2 раза?
 - Определите все пары столбцов матрицы M , сумма элементов которых больше K (например, $K = 75$).

```
[133]:  
# 10  
M = rand(1:10, 6, 10)  
  
[133]:  
6x10 Matrix{Int64}:  
 3 4 10 5 4 3 3 10 5 4  
 9 9 9 4 7 7 1 2 3 9  
 7 1 1 7 2 9 1 10 5 9  
 5 3 9 6 1 5 9 6 3 8  
 2 6 4 10 2 1 8 1 3 7  
 2 5 7 4 9 9 2 2 5 3
```

Рис. 2.33: Задание для самостоятельного выполнения №10

```
[142]:  
N = 4  
  
res1 = [count(x -> x > N, M[i, :]) for i in 1:size(M, 1)]  
res1  
  
[142]:  
6-element Vector{Int64}:  
 4  
 6  
 6  
 7  
 4  
 5  
  
[145]:  
L = 7  
  
res2 = findall(i -> count(x -> x == L, M[i,:]) == 2, 1:6)  
res2  
  
[145]:  
2-element Vector{Int64}:  
 2  
 3
```

Рис. 2.34: Задание для самостоятельного выполнения №10

```
[149]:
K = 75

res3 = [(i, j) for i in range(1, 6), j in range(2, 5) if (i != j and sum(M[:, i] + M[:, j]) > K)]

res3
```

```
[149]:
2-element Vector{Tuple{Int64, Int64}}:
 (4, 3)
 (3, 4)
```

Рис. 2.35: Задание для самостоятельного выполнения №10

11. Вычислите:

$$- \sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{(3+j)},$$

$$- \sum_{i=1}^{20} \sum_{j=1}^5 \frac{i^4}{(3+ij)}.$$

```
[152]:
# 11

sum([(i^4)/(3+j) for j in range(1, 20) for j in range(1, 5)])
```

```
[152]:
639215.2833333333
```

```
[153]:
sum([(i^4)/(3+j*j) for j in range(1, 20) for j in range(1, 5)])
```

```
[153]:
14632.78195488722
```

Рис. 2.36: Задание для самостоятельного выполнения №11

3 Выводы

В результате выполнения работы было освоено применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.